

Project Blueprint: AI Lecture Companion

Cybernauts 2026 — "Idea to Impact" (Programming & Tech)

1. One-Line Pitch

An AI-powered lecture companion that turns raw audio/video recordings into timestamp-linked study notes and an agentic tutor chatbot — designed around research on how students actually review lecture content.

Working name ideas (pick one and stick with it for branding): LectureLens, EchoNotes, RecallAI, ClassMind.

2. Why This Fits the Judging Criteria

Criterion	Weight	How this project addresses it
Research steps & project content	60%	Background research on note-taking/retention, survey of student pain points, gap analysis of existing tools, documented design rationale, evaluation of system performance
Speech delivery	10%	Clear problem→solution narrative, rehearsed demo flow
Visual outlook / workability	10%	Working live demo + poster/slides showing research→design→build→results flow
Teamwork & presentation	10%	Clear task ownership, each member can speak to their part
Q&A	10%	Documented design decisions ready to defend (why RAG, why agentic, evaluation results)

Key shift from a typical hackathon pitch: don't just show the app — show the *process* that led to it. Every major design decision should be traceable to either a research finding, a survey result, or a gap in existing tools.

3. Problem Statement (Research-Backed Framing)

Build this section using:

- 3-5 citations on lecture retention / note-taking effectiveness / cognitive load during lectures (quick Google Scholar search, abstracts are enough)
- A short survey (Google Form, 5 questions, sent to classmates tonight) covering: how students currently take notes, whether they rewatch recordings, what's frustrating about reviewing long lectures, whether they'd use an AI study tool
- Gap analysis: what Otter.ai / Notion AI / similar tools do NOT do (no timestamp-linked notes, no subject-specific tutoring, no citation-backed Q&A, etc.)

Target narrative: "Students lose information while simultaneously listening and writing notes [cite]. Existing transcription tools summarize but don't help students *navigate back* to specific explanations or *test* their understanding. Our survey of N students found X% struggle with reviewing long recordings and Y% want an interactive study tool — directly motivating our design."

4. Solution Overview

Core Pipeline

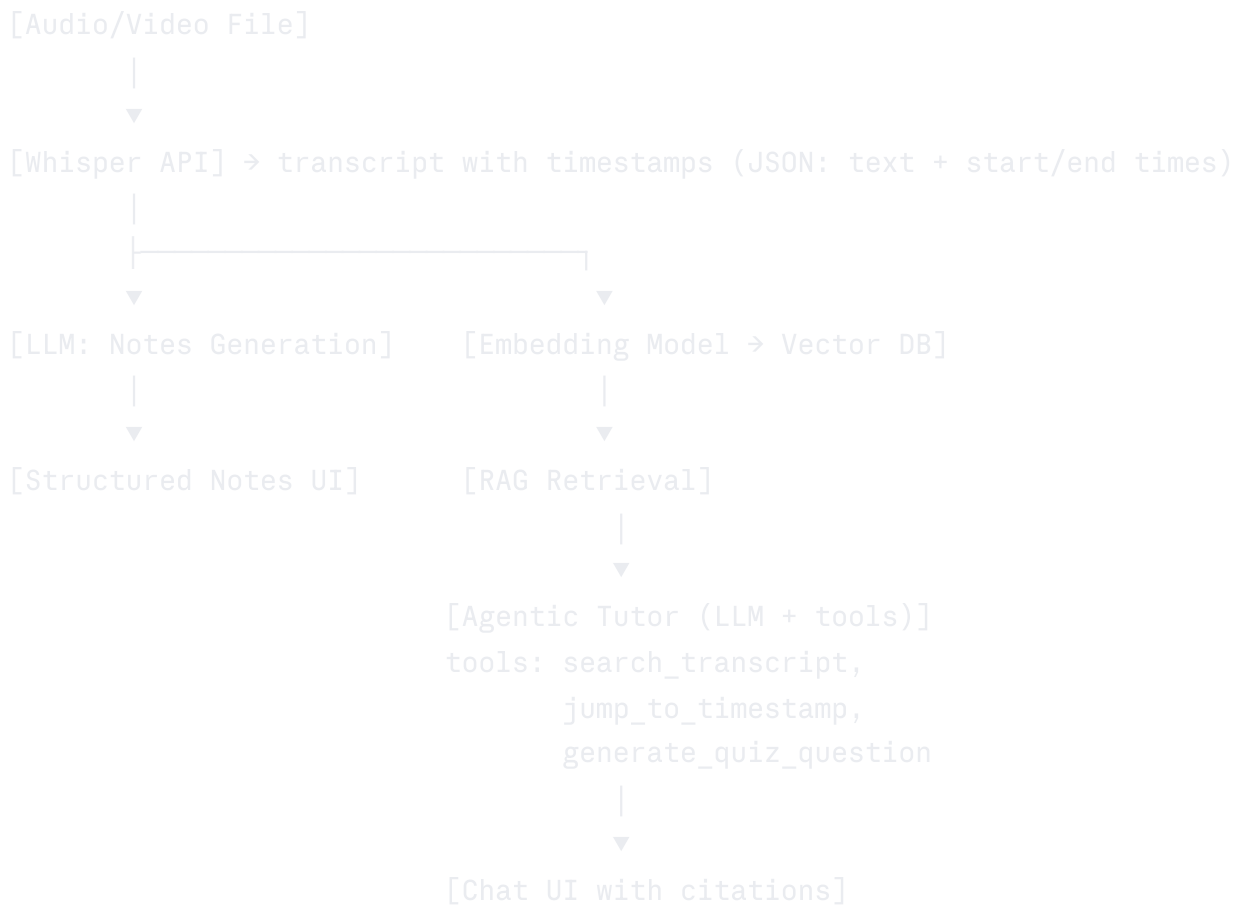
1. **Upload** — audio/video lecture recording
2. **Transcription** — speech-to-text with timestamps (Whisper API)
3. **Structured Notes Generation** — LLM converts transcript into topic-organized notes, each section linked to a timestamp
4. **RAG Indexing** — transcript chunks embedded into a vector store for retrieval
5. **Agentic Tutor Chatbot** — answers questions using retrieved transcript context, cites timestamps, can jump the video player to that moment, and can generate a quiz question on request

The "Research-Informed Differentiator"

Pick based on team strengths (see Section 6 for scope tiers):

- **Timestamp-linked notes** (click a note → jump to that moment in the recording) — directly addresses the "reviewing long recordings" pain point from your survey
 - **Agentic tutor with tools** (search transcript, jump timestamp, generate quiz) — addresses the "want interactive study help" pain point
-

5. Technical Architecture



Suggested stack:

- Transcription: Whisper API (or local whisper.cpp as backup if internet is unreliable at venue)
- LLM: Claude or GPT API (function/tool calling for the agent)
- Vector store: Chroma (simple, runs locally, no setup overhead) or in-memory cosine similarity if team is short on time
- Frontend: React (split-screen player + notes/chat panels)
- Backend: Lightweight Python (FastAPI) or Node, whichever the team is faster with

6. Feature Scope — Build in Tiers

Tier 1 — MUST HAVE (target: done by hour ~24-28)

- Upload audio/video
- Transcription with timestamps
- Structured notes generation (topic-organized)

- Basic RAG chatbot with citation back to transcript timestamp
- Clean, functional frontend connecting all of the above

→ If nothing else gets built, this alone is a complete, demoable, research-aligned project.

Tier 2 — SHOULD HAVE (target: hour ~24-44) — pick ONE

- **Option A:** Timestamp-linked notes (click note section → video jumps to that moment)
- **Option B:** Agentic tutor with 2-3 tools (search transcript, jump timestamp, generate quiz question)

Recommendation: Option B if your survey shows students want interactive help; Option A if it shows the bigger pain point is reviewing recordings. Either way, the timestamp data structure should be built into your pipeline from hour 1 regardless of which you pick.

Tier 3 — NICE TO HAVE (only if Tier 2 finished early)

- Flashcard/quiz export
- Key-moment highlighting (detect repeated/emphasized points)
- Multi-lecture support

CUT unless someone finishes early

- Slide/visual OCR (multimodal notes)
- Web-search fallback tool
- Cross-lecture concept linking
- Local model deployment

7. Research Workstream (runs in parallel with build)

Assign to whoever is least needed on core build at a given time — this does NOT need to block development.

Task	Time est.	Output
Literature scan (3-5 sources on retention/note-taking)	30-45 min	Citation list + 2-3 sentence summaries
Student survey (Google Form, send tonight)	20 min to create, runs overnight	Chart of responses
Gap analysis of existing tools (Otter.ai, Notion AI, etc.)	30-60 min	Comparison table
Design rationale log (ongoing)	Running doc	"Why we chose X over Y" for each major decision
System evaluation	1-2 hrs near the end	Test transcript accuracy on sample; test chatbot on 10 prepared Qs, record accuracy + citation correctness

8. Rough Timeline (Mon 1am → Wed 11am, ~58 hrs)

Time block	Focus
Hour 0-2 (Mon, now)	Finalize scope, send survey, set up repo/API keys, assign roles
Hour 2-12	Build transcription pipeline + notes generation (Tier 1)
Hour 12-24	Build RAG retrieval + basic chatbot; frontend skeleton
Hour 24-36	Integrate end-to-end Tier 1; start Tier 2 feature
Hour 36-46	Finish Tier 2; polish UI; run evaluation tests
Hour 46-52	Research workstream consolidation (survey results, gap analysis, citations) → poster/slides
Hour 52-56	Full rehearsal, fix bugs, prepare backup demo recording
Hour 56-58	Buffer / sleep / final setup

(Adjust based on actual sleep schedule — build in real rest, exhausted teams make more mistakes during judging.)

9. Presentation Structure (Poster / Slides + Demo)

1. **Problem** — research-backed: lecture review is hard, students lose info, cite literature + survey data
2. **Existing Solutions & Gap** — comparison table, what's missing
3. **Our Approach** — pipeline diagram + design rationale (why RAG, why agentic, why timestamp-linking)
4. **Live Demo** — pre-processed sample lecture: show notes page (timestamp jump), then chatbot (factual Q with citation, quiz request, edge-case question)
5. **Evaluation/Results** — transcription accuracy, chatbot accuracy on test questions
6. **Future Work** — Tier 3 features, multi-lecture support, etc.

Demo safety net: record a working demo video as backup in case of live failure (wifi, API rate limits, etc.)

10. Team Role Suggestions (adjust to actual skills)

- **Person A:** Transcription pipeline + timestamp data structure
- **Person B:** Notes generation (LLM prompting) + Tier 2 feature backend
- **Person C:** RAG/vector store + agentic chatbot
- **Person D:** Frontend (player + notes + chat UI) + research workstream coordination

Integrate early and often — get a rough end-to-end flow working with placeholder data within the first 12-16 hours.